Modul Praktikum: CRUD SIMRS dengan React Native

Mata Kuliah: Pengembangan Aplikasi Mobile | **Sub-CPMK:** 5, 6, 7 (CPMK3 & CPMK4)

Studi Kasus: Frontend SIMRS: **Bagian A** master data poli; **Bagian B** dokter, pasien, pendaftaran, rekam medis, dan resep obat

Estimasi waktu: 6 × 50 menit (Bagian A: 2 × 50 menit, Bagian B: 4 × 50 menit)

1. Tujuan Praktikum

Setelah praktikum, mahasiswa mampu:

1. Membangun aplikasi React Native (Expo) dengan navigasi antar layar.
2. Menghubungkan aplikasi ke RESTful API PHP menggunakan `fetch`.
3. Mengimplementasikan operasi **CRUD** (Create, Read, Update, Delete) serta validasi form.
4. Menjalankan aplikasi yang sama di **browser (web)** maupun perangkat mobile dengan kode lintas platform.
5. Memodelkan relasi antartabel (kunci asing, `RESTRICT`, `CASCADE`, kendala unik) ke dalam alur aplikasi.
6. Menyusun komponen **generik** (layar daftar dan form berbasis konfigurasi) sehingga beberapa tabel dapat dibuat tanpa menulis ulang kode.

2. Dasar Teori Singkat

- **React Native**: framework JavaScript untuk membuat aplikasi Android/iOS dari satu basis kode (analogi: satu resep, dua jenis hidangan).
- **Expo**: perangkat bantu yang menyederhanakan pembuatan, menjalankan, dan menguji proyek React Native.
- **State (`useState`)**: data yang bila berubah akan membuat tampilan diperbarui otomatis.
- **REST API**: "pelayan restoran" antara aplikasi (pelanggan) dan database (dapur); aplikasi meminta data lewat URL.
- **Studi kasus industri**: aplikasi pendaftaran pasien rumah sakit menampilkan daftar poli (Umum, Gigi, Anak) dari server agar pasien dapat memilih tujuan kunjungan.

3. Ringkasan Endpoint API

Basis URL: `http://localhost/api-simrs`

Operasi	Endpoint	Method	Parameter
Read (semua)	`/poli/index.php`	GET	-
Read (cari nama)	`/poli/index.php?id=%kata%`	GET	`id` diisi pola `LIKE` (hanya 1 baris dikembalikan)
Create	`/poli/create.php`	POST	JSON body: `nama_poli`, `lokasi`
Update	`/poli/update.php`	POST	Query string: `id`, `nama_poli`, `lokasi`
Delete	`/poli/delete.php`	POST	Query string: `id`

Format respons (dari `sendResponse`): `{ "status": "success|error", "message": "...", "data": ... }`

> **Catatan penting:** `update.php` dan `delete.php` membaca `$_GET`, sehingga parameter dikirim lewat URL, bukan body JSON. Kode di modul ini menyesuaikan hal tersebut.

4. Alat dan Bahan

Node.js LTS, VS Code, XAMPP (Apache + MySQL aktif, folder `api-simrs` di `htdocs`), browser (Chrome/Edge) untuk pengujian web, serta aplikasi **Expo Go** di ponsel atau emulator Android Studio (opsional).

5. Bagian A: Langkah Kerja CRUD Poli

Langkah 1 — Uji API

Buka `http://localhost/api-simrs/poli/index.php` di browser. Pastikan muncul JSON berisi daftar poli. Bila gagal, periksa XAMPP dan basis data sebelum lanjut.

Langkah 2 — Membuat Proyek

```
npx create-expo-app@latest simrs-poli --template blank
cd simrs-poli
npm install @react-navigation/native @react-navigation/native-stack
npx expo install react-native-screens react-native-safe-area-context
# Dependensi agar dapat berjalan di browser
npx expo install react-dom react-native-web @expo/metro-runtime
```

Langkah 3 — Struktur Folder

```
simrs-poli/
├── App.js
└── src/
    ├── config/api.js
    ├── services/poliService.js
    ├── utils/dialog.js
    └── screens/
        ├── PoliListScreen.js
        └── PoliFormScreen.js
```

Buat folder `src/config`, `src/services`, `src/utils`, `src/screens`.

Langkah 4 — Konfigurasi URL (src/config/api.js)

`localhost` pada emulator/ponsel menunjuk ke perangkat itu sendiri, bukan komputer Anda. Karena itu alamatnya harus disesuaikan. Pada **browser**, `localhost` dapat dipakai karena browser berjalan di komputer yang sama dengan XAMPP (`Platform.OS` bernilai `'web'`).

```
import { Platform } from 'react-native';

// Ganti 192.168.1.10 dengan IP LAN komputer Anda (cek: ipconfig / ifconfig)
const LAN_IP = '192.168.1.10';
const USE_REAL_DEVICE = false; // true hanya bila memakai Expo Go di ponsel fisik

const HOST = USE_REAL_DEVICE
  ? LAN_IP
  : Platform.OS === 'android' ? '10.0.2.2' : 'localhost';

// Bila Apache memakai port lain (mis. 8080), tulis: `http://${HOST}:8080/api-simrs`
```

```
export const BASE_URL = `http://${HOST}/api-simrs`;
```

Untuk pengujian di browser, biarkan `USE_REAL_DEVICE = false`. Ubah ke `true` hanya saat memakai ponsel fisik (ponsel dan komputer harus satu jaringan Wi-Fi).

Langkah 5 — Lapisan Layanan API (src/services/poliService.js)

Semua pemanggilan API dikumpulkan di satu berkas agar layar tetap sederhana (prinsip *separation of concerns*).

```
import { BASE_URL } from '../config/api';

async function request(url, options) {
  const res = await fetch(url, options);
  const json = await res.json();
  if (!res.ok || json.status !== 'success') {
    throw new Error(json.message || 'Terjadi kesalahan pada server');
  }
  return json;
}

const q = encodeURIComponent;

// READ - semua poli
export const getPoli = async () =>
  (await request(`${BASE_URL}/poli/index.php`)).data;

// READ - cari berdasarkan nama (API mengembalikan 1 objek)
export const searchPoli = async (kata) =>
  (await request(`${BASE_URL}/poli/index.php?id=${q('%' + kata + '%')}`)).data;

// CREATE - body JSON
export const createPoli = (nama_poli, lokasi) =>
  request(`${BASE_URL}/poli/create.php`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ nama_poli, lokasi }),
  });

// UPDATE - parameter lewat query string
export const updatePoli = (id, nama_poli, lokasi) =>
  request(
    `${BASE_URL}/poli/update.php?id=${id}&nama_poli=${q(nama_poli)}&lokasi=${q(lokasi)}`,
    { method: 'POST' }
  );

// DELETE - parameter lewat query string
export const deletePoli = (id) =>
  request(`${BASE_URL}/poli/delete.php?id=${id}`, { method: 'POST' });
```

Langkah 6 — Helper Dialog Lintas Platform (src/utils/dialog.js)

`Alert.alert` bawaan React Native **tidak berfungsi di browser**: dialog tidak muncul dan tombolnya tidak pernah menjalankan aksi. Karena itu dibuat helper yang memilih cara tampil sesuai platform: `window.alert`/`window.confirm` di web, dan `Alert.alert` di Android/iOS.

```
import { Alert, Platform } from 'react-native';

// Pesan informasi; onOk dijalankan setelah pengguna menekan OK
export function showAlert(title, message, onOk) {
  if (Platform.OS === 'web') {
```

```
    window.alert(`${title}\n\n${message}`);
    if (onOk) onOk();
  } else {
    Alert.alert(title, message, [{ text: 'OK', onPress: onOk }]);
  }
}

// Dialog konfirmasi; onConfirm dijalankan bila pengguna menyetujui
export function confirmDialog(title, message, onConfirm) {
  if (Platform.OS === 'web') {
    if (window.confirm(`${title}\n\n${message}`)) onConfirm();
  } else {
    Alert.alert(title, message, [
      { text: 'Batal', style: 'cancel' },
      { text: 'Hapus', style: 'destructive', onPress: onConfirm },
    ]);
  }
}
```

Langkah 7 — Layar Daftar Poli (src/screens/PoliListScreen.js)

Fitur: menampilkan daftar (**Read**), pencarian, tarik-untuk-muat-ulang (di mobile), tombol edit, hapus dengan konfirmasi (**Delete**). Dialog memakai helper `showAlert` dan `confirmDialog` dari Langkah 6, bukan `Alert` langsung.

```
import React, { useCallback, useState } from 'react';
import {
  View, Text, FlatList, TextInput, TouchableOpacity,
  StyleSheet, ActivityIndicator,
} from 'react-native';
import { useFocusEffect } from '@react-navigation/native';
import { getPoli, searchPoli, deletePoli } from '../services/poliService';
import { showAlert, confirmDialog } from '../utils/dialog';

export default function PoliListScreen({ navigation }) {
  const [data, setData] = useState([]);
  const [loading, setLoading] = useState(false);
  const [keyword, setKeyword] = useState('');

  const loadData = async () => {
    setLoading(true);
    try {
      setData(await getPoli());
    } catch (e) {
      showAlert('Gagal memuat', e.message);
    } finally {
      setLoading(false);
    }
  };

  // Dipanggil setiap layar ini tampil kembali (mis. setelah simpan data)
  useFocusEffect(useCallback(() => { loadData(); }, []));

  const handleSearch = async () => {
    if (keyword.trim() === '') return loadData();
    setLoading(true);
    try {
      const hasil = await searchPoli(keyword.trim());
      setData([hasil]); // API mengembalikan satu objek
    } catch (e) {
      setData([]); // 404 = tidak ditemukan
    } finally {
      setLoading(false);
    }
  };
```

```
  const handleDelete = (item) => {
    confirmDialog('Hapus Poli', `Yakin menghapus "${item.nama_poli}"?`, async () => {
      try {
        await deletePoli(item.id);
        loadData();
      } catch (e) {
        showAlert('Gagal menghapus', e.message);
      }
    });
  };

  const renderItem = ({ item }) => (
    <View style={s.card}>
      <View style={{ flex: 1 }}>
        <Text style={s.title}>{item.nama_poli}</Text>
        <Text style={s.sub}>📍 {item.lokasi || '-'}</Text>
      </View>
      <TouchableOpacity
        style={[s.btn, { backgroundColor: '#f59e0b' }]}
        onPress={() => navigation.navigate('PoliForm', { poli: item })}>
        <Text style={s.btnText}>Edit</Text>
      </TouchableOpacity>
      <TouchableOpacity
        style={[s.btn, { backgroundColor: '#ef4444' }]}
        onPress={() => handleDelete(item)}>
        <Text style={s.btnText}>Hapus</Text>
      </TouchableOpacity>
    </View>
  );

  return (
    <View style={s.container}>
      <View style={s.searchRow}>
        <TextInput
          style={s.input}
          placeholder="Cari nama poli..."
          value={keyword}
          onChangeText={setKeyword}
          onSubmitEditing={handleSearch}
        />
        <TouchableOpacity style={[s.btn, { backgroundColor: '#2563eb' }]} onPress={handleSearch}>
          <Text style={s.btnText}>Cari</Text>
        </TouchableOpacity>
      </View>

      {loading && <ActivityIndicator size="large" color="#2563eb" />}

      <FlatList
        data={data}
        keyExtractor={(item) => String(item.id)}
        renderItem={renderItem}
        refreshing={loading}
        onRefresh={loadData}
        ListEmptyComponent={!loading && <Text style={s.empty}>Data poli tidak ditemukan</Text>}
      />

      <TouchableOpacity style={s.fab} onPress={() => navigation.navigate('PoliForm')}>
        <Text style={s.fabText}>＋</Text>
      </TouchableOpacity>
    </View>
  );
}

const s = StyleSheet.create({
  container: { flex: 1, padding: 12, backgroundColor: '#f3f4f6' },
  searchRow: { flexDirection: 'row', marginBottom: 10 },
  input: {
```

```
    flex: 1, backgroundColor: '#fff', borderRadius: 8,
    paddingHorizontal: 12, marginRight: 8, borderWidth: 1, borderColor: '#d1d5db',
  },
  card: {
    flexDirection: 'row', alignItems: 'center', backgroundColor: '#fff',
    padding: 14, borderRadius: 10, marginBottom: 10, elevation: 2,
  },
  title: { fontSize: 16, fontWeight: 'bold' },
  sub: { color: '#6b7280', marginTop: 2 },
  btn: { paddingVertical: 8, paddingHorizontal: 12, borderRadius: 6, marginLeft: 6 },
  btnText: { color: '#fff', fontWeight: '600' },
  empty: { textAlign: 'center', marginTop: 30, color: '#6b7280' },
  fab: {
    position: 'absolute', right: 20, bottom: 24, width: 56, height: 56,
    borderRadius: 28, backgroundColor: '#16a34a', alignItems: 'center',
    justifyContent: 'center', elevation: 5,
  },
  fabText: { color: '#fff', fontSize: 28, marginTop: -2 },
});
```

Langkah 8 — Layar Form Tambah/Ubah (src/screens/PoliFormScreen.js)

Satu layar dipakai untuk **Create** dan **Update**. Mode ditentukan oleh ada tidaknya parameter `poli`.

```
import React, { useState } from 'react';
import {
  View, Text, TextInput, TouchableOpacity, StyleSheet, ActivityIndicator,
} from 'react-native';
import { createPoli, updatePoli } from '../services/poliService';
import { showAlert } from '../utils/dialog';

export default function PoliFormScreen({ route, navigation }) {
  const poli = route.params?.poli;      // undefined = mode tambah
  const isEdit = !!poli;

  const [namaPoli, setNamaPoli] = useState(poli?.nama_poli ?? '');
  const [lokasi, setLokasi] = useState(poli?.lokasi ?? '');
  const [error, setError] = useState('');
  const [saving, setSaving] = useState(false);

  const handleSave = async () => {
    // Validasi input (Sub-CPMK 6)
    if (namaPoli.trim() === '') {
      setError('Nama poli wajib diisi');
      return;
    }
    setError('');
    setSaving(true);
    try {
      const res = isEdit
        ? await updatePoli(poli.id, namaPoli.trim(), lokasi.trim())
        : await createPoli(namaPoli.trim(), lokasi.trim());
      // goBack dijalankan setelah OK ditekan (berlaku di web maupun mobile)
      showAlert('Berhasil', res.message, () => navigation.goBack());
    } catch (e) {
      showAlert('Gagal menyimpan', e.message);
    } finally {
      setSaving(false);
    }
  };

  return (
    <View style={s.container}>
      <Text style={s.label}>Nama Poli *</Text>
      <TextInput
        style={[s.input, error && s.inputError]}
        placeholder="Contoh: Poli Gigi"
```

```
        value={namaPoli}
        onChangeText={setNamaPoli}
      />
      {!!error && <Text style={s.error}>{error}</Text>}

      <Text style={s.label}>Lokasi</Text>
      <TextInput
        style={s.input}
        placeholder="Contoh: Gedung A Lantai 1"
        value={lokasi}
        onChangeText={setLokasi}
      />

      <TouchableOpacity style={s.btn} onPress={handleSave} disabled={saving}>
        {saving
          ? <ActivityIndicator color="#fff" />
          : <Text style={s.btnText}>{isEdit ? 'Perbarui' : 'Simpan'}</Text>}
      </TouchableOpacity>
    </View>
  );
}

const s = StyleSheet.create({
  container: { flex: 1, padding: 16, backgroundColor: '#fff' },
  label: { fontWeight: '600', marginTop: 14, marginBottom: 6 },
  input: { borderWidth: 1, borderColor: '#d1d5db', borderRadius: 8, padding: 10 },
  inputError: { borderColor: '#ef4444' },
  error: { color: '#ef4444', marginTop: 4 },
  btn: { backgroundColor: '#2563eb', padding: 14, borderRadius: 8, marginTop: 24, alignItems: 'center'
},
  btnText: { color: '#fff', fontWeight: 'bold', fontSize: 16 },
});
```

Langkah 9 — Navigasi (App.js)

```
import React from 'react';
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import PoliListScreen from './src/screens/PoliListScreen';
import PoliFormScreen from './src/screens/PoliFormScreen';

const Stack = createNativeStackNavigator();

export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator>
        <Stack.Screen name="PoliList" component={PoliListScreen} options={{ title: 'Data Poli' }} />
        <Stack.Screen
          name="PoliForm"
          component={PoliFormScreen}
          options={({ route }) => ({
            title: route.params?.poli ? 'Ubah Poli' : 'Tambah Poli',
          })}
        />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

> **Catatan:** berkas `App.js` ini akan diganti pada Langkah 18 (Bagian B) ketika menu utama dan layar tabel lain ditambahkan. Bila hanya mengerjakan Bagian A, pakai versi ini.

Langkah 10 — Menjalankan Aplikasi

A. Di browser (disarankan untuk praktikum)

```
npx expo start --web
```

Atau jalankan `npx expo start`, lalu tekan **`w`**. Browser terbuka di `http://localhost:8081` (port dapat berbeda, lihat pesan di terminal). Tekan **F12** untuk membuka DevTools: tab **Network** menampilkan request dan respons API, tab **Console** menampilkan error JavaScript.

B. Di ponsel atau emulator

```
npx expo start
```

Pindai kode QR dengan Expo Go (ponsel), atau tekan `a` untuk emulator Android. Pastikan `USE_REAL_DEVICE` di `api.js` sudah sesuai.

> **Catatan web:** gestur tarik-untuk-muat-ulang (`onRefresh`) tidak tersedia di browser; gunakan tombol refresh browser atau tombol Cari dengan kolom kosong untuk memuat ulang data.

6. Skenario Pengujian Bagian A

No	Skenario	Hasil yang diharapkan
1	Buka aplikasi	Daftar poli tampil
2	Tambah poli dengan nama kosong	Pesan "Nama poli wajib diisi"
3	Tambah "Poli Mata"	Alert berhasil, data muncul di daftar
4	Edit lokasi "Poli Mata"	Data berubah setelah kembali ke daftar
5	Hapus "Poli Mata"	Dialog konfirmasi, lalu data hilang
6	Cari "Gigi"	Poli yang cocok tampil
7	Cari kata yang tidak ada	"Data poli tidak ditemukan"
8	(Browser) Tekan Hapus lalu batalkan dialog	Data tidak terhapus
9	(Browser) Simpan data, tekan OK pada dialog	Kembali ke daftar, data baru tampil
10	(Browser) Buka DevTools > Network saat menyimpan	Request `create.php` berstatus 201

7. Pemecahan Masalah Bagian A

Gejala	Penyebab	Solusi
`Network request failed`	Memakai `localhost` di perangkat	Gunakan IP LAN (ponsel) atau `10.0.2.2` (emulator Android)
Tetap gagal terhubung	Firewall memblokir Apache	Izinkan port 80 pada firewall
`JSON Parse error`	Server mengembalikan error PHP berbentuk HTML	Buka URL di browser untuk melihat pesan error
Edit tidak mengubah data	Parameter dikirim lewat body JSON	Gunakan query string (lihat Langkah 5)

Gejala	Penyebab	Solusi
`blocked by CORS policy` (browser)	Error PHP muncul sebelum header CORS terkirim	Buka URL API langsung di browser, perbaiki error PHP, pastikan tidak ada spasi sebelum `<?php`
Tombol Hapus tidak bereaksi (browser)	Masih memakai `Alert.alert`	Gunakan `confirmDialog` (Langkah 6 dan 7)
Setelah Simpan layar tidak kembali (browser)	Callback `Alert.alert` tidak berjalan di web	Gunakan `showAlert` dengan callback (Langkah 8)
Halaman web kosong	Dependensi web belum terpasang	Jalankan perintah Langkah 2, lalu `npx expo start --web --clear`
`Failed to fetch` (browser)	Apache mati atau port pada `BASE_URL` salah	Periksa XAMPP dan `BASE_URL`
Pencarian hanya menampilkan 1 hasil	`index.php` memakai `fetch()`	Keterbatasan API; ubah ke `fetchAll()` bila diperlukan

8. Bagian B: Praktikum Lanjutan (Dokter, Pasien, Pendaftaran, Rekam Medis, Resep Obat)

Waktu: 4 × 50 menit. **Prasyarat:** Bagian A (Poli) selesai dan berjalan. Pada bagian ini mahasiswa membangun CRUD untuk lima tabel berelasi dengan **komponen generik**: satu pabrik layar daftar dan satu pabrik layar form yang dikonfigurasi per tabel (prinsip DRY: *Don't Repeat Yourself*). Analogi: satu cetakan kue dengan lima isi berbeda.

Langkah 11 — Memahami Skema dan Relasi

Tabel	Kunci Asing	Aturan Hapus	Kendala Unik
pasien	user_id → users.id	CASCADE	user_id, nik, no_rm
dokter	tidak ada	tidak ada	no_sip
pendaftaran	pasien_id → pasien.id; jadwal_id → jadwal_praktik.id	RESTRICT	jadwal_id + tanggal_kunjungan + no_antrian
rekam_medis	pendaftaran_id → pendaftaran.id; dokter_id → dokter.id	RESTRICT	pendaftaran_id (relasi 1 banding 1)
resep_obat	rekam_medis_id → rekam_medis.id	CASCADE	tidak ada

Alur data: pasien mendaftar (pendaftaran), diperiksa dokter (rekam_medis), lalu dokter menulis banyak resep (resep_obat).

- **RESTRICT**: data induk tidak bisa dihapus selama masih dipakai tabel anak (mis. dokter yang sudah punya rekam medis).
- **CASCADE**: hapus induk, anak ikut terhapus (mis. hapus rekam medis menghapus semua resepnya).

Urutan pengisian data (wajib mengikuti kunci asing):

1. users, lalu pasien (butuh `user_id` yang sudah ada).
2. dokter dan poli, lalu jadwal_praktik (butuh dokter dan poli).
3. pendaftaran (butuh pasien dan jadwal_praktik).
4. rekam_medis (butuh pendaftaran dan dokter).

5. resep_obat (butuh rekam_medis).

Langkah 12 — Kontrak Endpoint yang Diasumsikan

Modul ini mengasumsikan setiap tabel punya folder API dengan pola yang sama seperti `poli` (index, create, update, delete). `create` menerima body JSON, `update` dan `delete` menerima parameter lewat query string, dan `index` mengembalikan `data` berupa array.

Sumber Data	Folder API	Field yang Dikirim
dokter	/dokter/	nama, spesialisasi, no_sip
pasien	/pasien/	user_id, nik, no_rm, nama, tgl_lahir, jenis_kelamin, alamat
pendaftaran	/pendaftaran/	pasien_id, jadwal_id, tanggal_kunjungan, no_antrian, status
rekam_medis	/rekam_medis/	pendaftaran_id, dokter_id, diagnosa, catatan
resep_obat	/resep_obat/	rekam_medis_id, nama_obat, dosis, jumlah

> **Catatan:** bila `update.php` pada API Anda membaca body JSON, ubah `createCrudService('nama', { updateVia: 'json' })` pada Langkah 14. Daftar resep difilter di sisi aplikasi berdasarkan `rekam_medis_id`, sehingga tidak bergantung pada fitur filter di API.

Struktur folder tambahan:

```
src/
├── components/
│   ├── makeListScreen.js
│   └── makeFormScreen.js
├── services/
│   ├── crudService.js
│   └── apis.js
├── utils/validators.js
└── screens/
    ├── HomeScreen.js
    └── entityScreens.js
```

Langkah 13 — Validator (src/utils/validators.js)

Setiap validator menerima nilai dan mengembalikan pesan error (string kosong bila valid).

```
export const required = (label) => (v) =>
  String(v ?? '').trim() === '' ? `${label} wajib diisi` : '';

export const exactDigits = (label, n) => (v) =>
  /^\d+$/.test(v) && v.length === n ? '' : `${label} harus ${n} digit angka`;

export const positiveInt = (label) => (v) =>
  /^\d+$/.test(String(v)) && Number(v) > 0 ? '' : `${label} harus bilangan bulat lebih dari 0`;

export const isDate = (label) => (v) => {
  if (!/^\d{4}-\d{2}-\d{2}$/.test(v)) return `${label} berformat YYYY-MM-DD`;
  const d = new Date(v);
  return isNaN(d) || d.toISOString().slice(0, 10) !== v
    ? `${label} bukan tanggal yang valid` : '';
};
```

Penting: **NIK disimpan sebagai string**, bukan angka. Angka JavaScript kehilangan presisi di atas 15 digit dan menghapus angka nol di depan.

Langkah 14 — Layanan CRUD Generik (src/services/crudService.js)

```
import { BASE_URL } from '../config/api';

const q = encodeURIComponent;

async function request(url, options) {
  const res = await fetch(url, options);
  let json;
  try {
    json = await res.json();
  } catch (e) {
    throw new Error('Respons server bukan JSON. Periksa error PHP.');
  }
  if (!res.ok || json.status !== 'success') {
    throw new Error(json.message || 'Terjadi kesalahan pada server');
  }
  return json;
}

const toQuery = (obj) =>
  Object.entries(obj).map(([k, v]) => `${k}=${q(v ?? '')}`).join('&');

const jsonOpts = (body) => ({
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify(body),
});

// updateVia: 'query' (seperti poli/update.php) atau 'json'
export function createCrudService(resource, { updateVia = 'query' } = {}) {
  const url = (file, qs = '') => `${BASE_URL}/${resource}/${file}.php${qs}`;
  return {
    list: async () => {
      const data = (await request(url('index'))).data;
      return Array.isArray(data) ? data : [];
    },
    create: (body) => request(url('create'), jsonOpts(body)),
    update: (id, body) =>
      updateVia === 'json'
        ? request(url('update'), jsonOpts({ id, ...body }))
        : request(url('update', `?${toQuery({ id, ...body })}`), { method: 'POST' }),
    remove: (id) => request(url('delete', `?id=${id}`), { method: 'POST' }),
  };
}
```

src/services/apis.js:

```
import { createCrudService } from './crudService';

export const dokterApi = createCrudService('dokter');
export const pasienApi = createCrudService('pasien');
export const pendaftaranApi = createCrudService('pendaftaran');
export const rekamMedisApi = createCrudService('rekam_medis');
export const resepApi = createCrudService('resep_obat');
```

Langkah 15 — Pabrik Layar Daftar (src/components/makeListScreen.js)

Parameter konfigurasi: `api`, `formRoute`, `titleOf`, `subOf`, `filterBy` (opsional, untuk data anak), dan `actions` (tombol tambahan per baris).

```
import React, { useCallback, useState } from 'react';
import {
```

```
  View, Text, FlatList, TouchableOpacity, StyleSheet, ActivityIndicator,
} from 'react-native';
import { useFocusEffect } from '@react-navigation/native';
import { showAlert, confirmDialog } from '../utils/dialog';

export default function makeListScreen({ api, formRoute, titleOf, subOf, filterBy, actions }) {
  return function ListScreen({ navigation, route }) {
    const filterValue = filterBy ? route.params?.[filterBy] : undefined;
    const defaults = filterBy ? { [filterBy]: String(filterValue) } : undefined;
    const [data, setData] = useState([]);
    const [loading, setLoading] = useState(false);

    const loadData = async () => {
      setLoading(true);
      try {
        let rows = await api.list();
        if (filterBy) rows = rows.filter((r) => String(r[filterBy]) === String(filterValue));
        setData(rows);
      } catch (e) {
        showAlert('Gagal memuat', e.message);
      } finally {
        setLoading(false);
      }
    };

    useFocusEffect(useCallback(() => { loadData(); }, []));

    const handleDelete = (item) =>
      confirmDialog('Hapus Data', `Yakin menghapus "${titleOf(item)}"?`, async () => {
        try {
          await api.remove(item.id);
          loadData();
        } catch (e) {
          showAlert('Gagal menghapus', e.message); // mis. ditolak karena relasi RESTRICT
        }
      });

    const renderItem = ({ item }) => {
      const extra = actions ? actions(item, { reload: loadData, navigation }) : [];
      const buttons = [
        { label: 'Edit', color: '#f59e0b', onPress: () => navigation.navigate(formRoute, { item,
defaults }) },
        ...extra,
        { label: 'Hapus', color: '#ef4444', onPress: () => handleDelete(item) },
      ];
      return (
        <View style={s.card}>
          <Text style={s.title}>{titleOf(item)}</Text>
          <Text style={s.sub}>{subOf(item)}</Text>
          <View style={s.btnRow}>
            {buttons.map((b) => (
              <TouchableOpacity key={b.label} style={[s.btn, { backgroundColor: b.color }]}
onPress={b.onPress}>
                <Text style={s.btnText}>{b.label}</Text>
              </TouchableOpacity>
            ))}
          </View>
        </View>
      );
    };

    return (
      <View style={s.container}>
        {loading && <ActivityIndicator size="large" color="#2563eb" />}
        <FlatList
          data={data}
          keyExtractor={(item) => String(item.id)}
          renderItem={renderItem}
          refreshing={loading}
```

```
          onRefresh={loadData}
          ListEmptyComponent={!loading && <Text style={s.empty}>Belum ada data</Text>}
        />
        <TouchableOpacity style={s.fab} onPress={() => navigation.navigate(formRoute, { defaults })}>
          <Text style={s.fabText}>+</Text>
        </TouchableOpacity>
      </View>
    );
  };
}

const s = StyleSheet.create({
  container: { flex: 1, padding: 12, backgroundColor: '#f3f4f6' },
  card: { backgroundColor: '#fff', padding: 14, borderRadius: 10, marginBottom: 10, elevation: 2 },
  title: { fontSize: 16, fontWeight: 'bold' },
  sub: { color: '#6b7280', marginTop: 2 },
  btnRow: { flexDirection: 'row', flexWrap: 'wrap', marginTop: 10 },
  btn: { paddingVertical: 7, paddingHorizontal: 12, borderRadius: 6, marginRight: 6, marginBottom: 4
},
  btnText: { color: '#fff', fontWeight: '600' },
  empty: { textAlign: 'center', marginTop: 30, color: '#6b7280' },
  fab: {
    position: 'absolute', right: 20, bottom: 24, width: 56, height: 56, borderRadius: 28,
    backgroundColor: '#16a34a', alignItems: 'center', justifyContent: 'center', elevation: 5,
  },
  fabText: { color: '#fff', fontSize: 28, marginTop: -2 },
});
```

Langkah 16 — Pabrik Layar Form (src/components/makeFormScreen.js)

Setiap field dikonfigurasi dengan: `name`, `label`, `type` (`text`, `number`, `date`, `multiline`, `select`, `hidden`), `required`, `maxLength`, `validate` (daftar validator), `options` (pilihan statis) atau `loadOptions` (pilihan dari API), dan `default`.

```
import React, { useEffect, useState } from 'react';
import {
  View, Text, TextInput, TouchableOpacity, ScrollView, StyleSheet, ActivityIndicator,
} from 'react-native';
import { showAlert } from '../utils/dialog';
import { required } from '../utils/validators';

export default function makeFormScreen({ api, fields }) {
  return function FormScreen({ route, navigation }) {
    const item = route.params?.item;           // ada = mode ubah
    const defaults = route.params?.defaults || {};
    const isEdit = !!item;

    const [values, setValues] = useState(() => {
      const v = {};
      fields.forEach((f) => {
        v[f.name] = String(item?.[f.name] ?? defaults[f.name] ?? f.default ?? '');
      });
      return v;
    });
    const [errors, setErrors] = useState({});
    const [options, setOptions] = useState({});
    const [saving, setSaving] = useState(false);

    // Muat pilihan untuk field select yang datanya berasal dari API
    useEffect(() => {
      fields.filter((f) => f.loadOptions).forEach(async (f) => {
        try {
          const opts = await f.loadOptions();
          setOptions((o) => ({ ...o, [f.name]: opts }));
        } catch (e) {
          showAlert('Gagal memuat pilihan', e.message);
```

```
      }
    });
  }, []);

  const setField = (name, val) => setValues((v) => ({ ...v, [name]: val }));

  const validate = () => {
    const err = {};
    fields.forEach((f) => {
      if (f.type === 'hidden') return;
      const val = values[f.name].trim();
      if (val === '' && !f.required) return;
      const rules = [...(f.required ? [required(f.label)] : []), ...(f.validate || [])];
      for (const rule of rules) {
        const msg = rule(val);
        if (msg) { err[f.name] = msg; break; }
      }
    });
    setErrors(err);
    return Object.keys(err).length === 0;
  };

  const handleSave = async () => {
    if (!validate()) return;
    setSaving(true);
    try {
      const body = {};
      fields.forEach((f) => { body[f.name] = values[f.name].trim(); });
      const res = isEdit ? await api.update(item.id, body) : await api.create(body);
      showAlert('Berhasil', res.message, () => navigation.goBack());
    } catch (e) {
      showAlert('Gagal menyimpan', e.message);
    } finally {
      setSaving(false);
    }
  };

  const renderField = (f) => {
    if (f.type === 'hidden') return null;
    const opts = options[f.name] || f.options || [];
    return (
      <View key={f.name}>
        <Text style={s.label}>{f.label}{f.required ? ' *' : ''}</Text>
        {f.type === 'select' ? (
          <View style={s.chipRow}>
            {opts.length === 0 && <Text style={s.hint}>Pilihan kosong / sedang dimuat...</Text>}
            {opts.map((o) => {
              const active = values[f.name] === String(o.value);
              return (
                <TouchableOpacity
                  key={o.value}
                  style={[s.chip, active && s.chipActive]}
                  onPress={() => setField(f.name, String(o.value))}>
                  <Text style={active ? s.chipTextActive : s.chipText}>{o.label}</Text>
                </TouchableOpacity>
              );
            })}
          </View>
        ) : (
          <TextInput
            style={[s.input, f.type === 'multiline' && s.multiline, errors[f.name] && s.inputError]}
            value={values[f.name]}
            onChangeText={(t) => setField(f.name, t)}
            placeholder={f.type === 'date' ? 'YYYY-MM-DD' : f.placeholder}
            keyboardType={f.type === 'number' ? 'numeric' : 'default'}
            multiline={f.type === 'multiline'}
            maxLength={f.maxLength}
          />
        )}
        {!!errors[f.name] && <Text style={s.error}>{errors[f.name]}</Text>}
```

```
        </View>
      );
    };

    return (
      <ScrollView style={s.container} keyboardShouldPersistTaps="handled">
        {fields.map(renderField)}
        <TouchableOpacity style={s.btn} onPress={handleSave} disabled={saving}>
          {saving ? <ActivityIndicator color="#fff" />
                  : <Text style={s.btnText}>{isEdit ? 'Perbarui' : 'Simpan'}</Text>}
        </TouchableOpacity>
        <View style={{ height: 40 }} />
      </ScrollView>
    );
  };
}

const s = StyleSheet.create({
  container: { flex: 1, padding: 16, backgroundColor: '#fff' },
  label: { fontWeight: '600', marginTop: 14, marginBottom: 6 },
  input: { borderWidth: 1, borderColor: '#d1d5db', borderRadius: 8, padding: 10 },
  multiline: { minHeight: 80, textAlignVertical: 'top' },
  inputError: { borderColor: '#ef4444' },
  error: { color: '#ef4444', marginTop: 4 },
  hint: { color: '#6b7280' },
  chipRow: { flexDirection: 'row', flexWrap: 'wrap' },
  chip: { borderWidth: 1, borderColor: '#d1d5db', borderRadius: 16, paddingVertical: 6,
paddingHorizontal: 12, marginRight: 6, marginBottom: 6 },
  chipActive: { backgroundColor: '#2563eb', borderColor: '#2563eb' },
  chipText: { color: '#111827' },
  chipTextActive: { color: '#fff', fontWeight: '600' },
  btn: { backgroundColor: '#2563eb', padding: 14, borderRadius: 8, marginTop: 24, alignItems: 'center'
},
  btnText: { color: '#fff', fontWeight: 'bold', fontSize: 16 },
});
```

Langkah 17 — Konfigurasi Layar Tiap Tabel (src/screens/entityScreens.js)

Inilah inti Bagian B: lima tabel hanya perlu konfigurasi, bukan menulis ulang layar.

```
import makeListScreen from '../components/makeListScreen';
import makeFormScreen from '../components/makeFormScreen';
import {
  dokterApi, pasienApi, pendaftaranApi, rekamMedisApi, resepApi,
} from '../services/apis';
import { exactDigits, isDate, positiveInt } from '../utils/validators';
import { showAlert } from '../utils/dialog';

// ---------- Pemuat pilihan untuk field select ----------
const pasienOptions = async () =>
  (await pasienApi.list()).map((p) => ({ value: p.id, label: `${p.no_rm} - ${p.nama}` }));
const dokterOptions = async () =>
  (await dokterApi.list()).map((d) => ({ value: d.id, label: `${d.nama} (${d.spesialisasi || '-'})`
}));
const pendaftaranOptions = async () =>
  (await pendaftaranApi.list()).map((p) => ({
    value: p.id, label: `#${p.id} antrian ${p.no_antrian} (${p.tanggal_kunjungan})`,
  }));

// ---------- DOKTER ----------
export const DokterList = makeListScreen({
  api: dokterApi, formRoute: 'DokterForm',
  titleOf: (d) => d.nama,
  subOf: (d) => `${d.spesialisasi || '-'} | SIP: ${d.no_sip || '-'}`,
});
export const DokterForm = makeFormScreen({
  api: dokterApi,
```

```
  fields: [
    { name: 'nama', label: 'Nama Dokter', required: true, maxLength: 100 },
    { name: 'spesialisasi', label: 'Spesialisasi', maxLength: 100 },
    { name: 'no_sip', label: 'No. SIP', maxLength: 50 },
  ],
});

// ---------- PASIEN ----------
export const PasienList = makeListScreen({
  api: pasienApi, formRoute: 'PasienForm',
  titleOf: (p) => p.nama,
  subOf: (p) => `RM ${p.no_rm} | NIK ${p.nik} | ${p.jenis_kelamin} | ${p.tgl_lahir}`,
});
export const PasienForm = makeFormScreen({
  api: pasienApi,
  fields: [
    { name: 'user_id', label: 'ID User (akun login)', type: 'number', required: true, validate:
[positiveInt('ID User')] },
    { name: 'nik', label: 'NIK', type: 'number', required: true, maxLength: 16, validate:
[exactDigits('NIK', 16)] },
    { name: 'no_rm', label: 'No. Rekam Medis', required: true, maxLength: 20 },
    { name: 'nama', label: 'Nama Lengkap', required: true, maxLength: 100 },
    { name: 'tgl_lahir', label: 'Tanggal Lahir', type: 'date', required: true, validate:
[isDate('Tanggal lahir')] },
    {
      name: 'jenis_kelamin', label: 'Jenis Kelamin', type: 'select', required: true,
      options: [{ value: 'L', label: 'Laki-laki' }, { value: 'P', label: 'Perempuan' }],
    },
    { name: 'alamat', label: 'Alamat', type: 'multiline' },
  ],
});

// ---------- PENDAFTARAN ----------
const STATUS_COLOR = { dipanggil: '#2563eb', selesai: '#16a34a', batal: '#6b7280' };

export const PendaftaranList = makeListScreen({
  api: pendaftaranApi, formRoute: 'PendaftaranForm',
  titleOf: (p) => `Antrian ${p.no_antrian} - ${p.tanggal_kunjungan}`,
  subOf: (p) => `Pasien #${p.pasien_id} | Jadwal #${p.jadwal_id} | Status: ${p.status}`,
  // Tombol ubah status (alur: menunggu -> dipanggil -> selesai / batal)
  actions: (item, { reload }) =>
    ['dipanggil', 'selesai', 'batal']
      .filter((st) => st !== item.status)
      .map((st) => ({
        label: st, color: STATUS_COLOR[st],
        onPress: async () => {
          try {
            await pendaftaranApi.update(item.id, {
              pasien_id: item.pasien_id, jadwal_id: item.jadwal_id,
              tanggal_kunjungan: item.tanggal_kunjungan, no_antrian: item.no_antrian,
              status: st,
            });
            reload();
          } catch (e) {
            showAlert('Gagal ubah status', e.message);
          }
        },
      })),
});
export const PendaftaranForm = makeFormScreen({
  api: pendaftaranApi,
  fields: [
    { name: 'pasien_id', label: 'Pasien', type: 'select', required: true, loadOptions: pasienOptions
},
    { name: 'jadwal_id', label: 'ID Jadwal Praktik', type: 'number', required: true, validate:
[positiveInt('ID Jadwal')] },
    { name: 'tanggal_kunjungan', label: 'Tanggal Kunjungan', type: 'date', required: true, validate:
[isDate('Tanggal kunjungan')] },
    { name: 'no_antrian', label: 'No. Antrian', type: 'number', required: true, validate:
[positiveInt('No. antrian')] },
```

```
    {
      name: 'status', label: 'Status', type: 'select', required: true, default: 'menunggu',
      options: ['menunggu', 'dipanggil', 'selesai', 'batal'].map((v) => ({ value: v, label: v })),
    },
  ],
});

// ---------- REKAM MEDIS ----------
export const RekamMedisList = makeListScreen({
  api: rekamMedisApi, formRoute: 'RekamMedisForm',
  titleOf: (r) => `Rekam Medis #${r.id} (Pendaftaran #${r.pendaftaran_id})`,
  subOf: (r) => `Dokter #${r.dokter_id} | Diagnosa: ${r.diagnosa}`,
  actions: (item, { navigation }) => [{
    label: 'Resep', color: '#7c3aed',
    onPress: () => navigation.navigate('ResepList', { rekam_medis_id: item.id }),
  }],
});
export const RekamMedisForm = makeFormScreen({
  api: rekamMedisApi,
  fields: [
    { name: 'pendaftaran_id', label: 'Pendaftaran', type: 'select', required: true, loadOptions:
pendaftaranOptions },
    { name: 'dokter_id', label: 'Dokter Pemeriksa', type: 'select', required: true, loadOptions:
dokterOptions },
    { name: 'diagnosa', label: 'Diagnosa', type: 'multiline', required: true },
    { name: 'catatan', label: 'Catatan', type: 'multiline' },
  ],
});

// ---------- RESEP OBAT (data anak dari rekam medis) ----------
export const ResepList = makeListScreen({
  api: resepApi, formRoute: 'ResepForm', filterBy: 'rekam_medis_id',
  titleOf: (r) => `${r.nama_obat} x${r.jumlah}`,
  subOf: (r) => `Dosis: ${r.dosis || '-'}`,
});
export const ResepForm = makeFormScreen({
  api: resepApi,
  fields: [
    { name: 'rekam_medis_id', type: 'hidden' },
    { name: 'nama_obat', label: 'Nama Obat', required: true, maxLength: 150 },
    { name: 'dosis', label: 'Dosis (mis. 3 x 1 sehari)', maxLength: 100 },
    { name: 'jumlah', label: 'Jumlah', type: 'number', required: true, default: '1', validate:
[positiveInt('Jumlah')] },
  ],
});
```

Langkah 18 — Menu Utama dan Navigasi

Buat `src/screens/HomeScreen.js`:

```
import React from 'react';
import { ScrollView, Text, TouchableOpacity, StyleSheet } from 'react-native';

const MENU = [
  { label: 'Data Poli', route: 'PoliList' },
  { label: 'Data Dokter', route: 'DokterList' },
  { label: 'Data Pasien', route: 'PasienList' },
  { label: 'Pendaftaran Antrian', route: 'PendaftaranList' },
  { label: 'Rekam Medis (dan Resep)', route: 'RekamMedisList' },
];

export default function HomeScreen({ navigation }) {
  return (
    <ScrollView style={s.container}>
      {MENU.map((m) => (
        <TouchableOpacity key={m.route} style={s.item} onPress={() => navigation.navigate(m.route)}>
          <Text style={s.text}>{m.label}</Text>
```

```
        </TouchableOpacity>
      ))}
    </ScrollView>
  );
}

const s = StyleSheet.create({
  container: { flex: 1, padding: 16, backgroundColor: '#f3f4f6' },
  item: { backgroundColor: '#fff', padding: 18, borderRadius: 10, marginBottom: 12, elevation: 2 },
  text: { fontSize: 16, fontWeight: '600' },
});
```

Ganti seluruh isi `App.js` (versi Langkah 9) dengan:

```
import React from 'react';
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import HomeScreen from './src/screens/HomeScreen';
import PoliListScreen from './src/screens/PoliListScreen';
import PoliFormScreen from './src/screens/PoliFormScreen';
import {
  DokterList, DokterForm, PasienList, PasienForm, PendaftaranList,
  PendaftaranForm, RekamMedisList, RekamMedisForm, ResepList, ResepForm,
} from './src/screens/entityScreens';

const Stack = createNativeStackNavigator();

const listOpt = (title) => ({ title });
const formOpt = (nama) => ({ route }) => ({
  title: `${route.params?.item ? 'Ubah' : 'Tambah'} ${nama}`,
});

export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator>
        <Stack.Screen name="Home" component={HomeScreen} options={listOpt('SIMRS')} />
        <Stack.Screen name="PoliList" component={PoliListScreen} options={listOpt('Data Poli')} />
        <Stack.Screen name="PoliForm" component={PoliFormScreen} options={formOpt('Poli')} />
        <Stack.Screen name="DokterList" component={DokterList} options={listOpt('Data Dokter')} />
        <Stack.Screen name="DokterForm" component={DokterForm} options={formOpt('Dokter')} />
        <Stack.Screen name="PasienList" component={PasienList} options={listOpt('Data Pasien')} />
        <Stack.Screen name="PasienForm" component={PasienForm} options={formOpt('Pasien')} />
        <Stack.Screen name="PendaftaranList" component={PendaftaranList}
options={listOpt('Pendaftaran')} />
        <Stack.Screen name="PendaftaranForm" component={PendaftaranForm}
options={formOpt('Pendaftaran')} />
        <Stack.Screen name="RekamMedisList" component={RekamMedisList} options={listOpt('Rekam
Medis')} />
        <Stack.Screen name="RekamMedisForm" component={RekamMedisForm} options={formOpt('Rekam
Medis')} />
        <Stack.Screen name="ResepList" component={ResepList} options={listOpt('Resep Obat')} />
        <Stack.Screen name="ResepForm" component={ResepForm} options={formOpt('Resep')} />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

Bila Anda masih memakai `PoliListScreen` dari Bagian A, tombol kembali ke menu utama otomatis tersedia dari header navigasi.

Langkah 19 — Pengujian Bagian B

Jalankan `npx expo start --web` (atau di ponsel), lalu uji **berurutan** sesuai urutan data pada Langkah 11.

No	Skenario	Hasil yang diharapkan
1	Tambah dokter tanpa nama	Pesan "Nama Dokter wajib diisi"
2	Tambah dokter lengkap, lalu edit spesialisasi	Data berubah di daftar
3	Tambah pasien dengan NIK 15 digit	Pesan "NIK harus 16 digit angka"
4	Tambah pasien dengan tanggal lahir 2000-02-31	Pesan "bukan tanggal yang valid"
5	Tambah pasien dengan NIK yang sudah ada	Pesan dari server (duplikat) tampil di dialog
6	Tambah pendaftaran, pilih pasien dari daftar pilihan	Data muncul, status "menunggu"
7	Tambah pendaftaran dengan jadwal, tanggal, dan nomor antrian sama	Ditolak server (kendala unik)
8	Tekan tombol "dipanggil" lalu "selesai" pada pendaftaran	Status pada kartu berubah
9	Tambah rekam medis untuk pendaftaran yang sama dua kali	Ditolak (relasi 1 banding 1)
10	Dari rekam medis, tekan "Resep", tambah 2 obat	Dua resep tampil pada layar Resep
11	Hapus pasien yang sudah punya pendaftaran	Ditolak (RESTRICT)
12	Hapus rekam medis yang punya resep	Rekam medis dan seluruh resepnya terhapus (CASCADE)

Pemecahan masalah Bagian B:

Gejala	Penyebab	Solusi
Pilihan pasien/dokter kosong	Tabel sumber belum berisi data atau API gagal	Isi data lebih dulu (Langkah 11); cek tab Network di DevTools
Daftar kosong padahal data ada	`index.php` tidak mengembalikan array pada `data`	Periksa respons JSON; sesuaikan API
Edit tidak mengubah data	`update.php` membaca body JSON, bukan query string	Gunakan `{ updateVia: 'json' }` pada `createCrudService`
Dialog "Cannot add or update a child row"	Kunci asing tidak valid (mis. `user_id` atau `jadwal_id` belum ada)	Isi tabel induk lebih dulu
Dialog "Cannot delete or update a parent row"	Data masih dipakai tabel anak (RESTRICT)	Hapus data anak lebih dulu
Dokter kedua tanpa No. SIP ditolak	String kosong dikirim, bukan NULL, sehingga melanggar kendala unik	Perbaiki API: ubah string kosong menjadi NULL
Daftar resep selalu kosong	Layar dibuka tanpa parameter `rekam_medis_id`	Buka Resep melalui tombol pada kartu Rekam Medis

9. Catatan Kualitas API (Bahan Diskusi)

1. `update.php` dan `delete.php` memakai `$_GET`: operasi mengubah data sebaiknya via body (POST/PUT/DELETE), karena URL tercatat di log dan mudah terpicu tidak sengaja.

2. Update dan delete mengembalikan kode `201` (Created); kode yang lebih tepat adalah `200`.
3. `Access-Control-Allow-Origin: *` cocok untuk praktikum, tetapi tidak aman untuk produksi.
4. Belum ada autentikasi `api_token` (tabel `users`) pada endpoint poli.
5. Untuk tabel Bagian B, field kosong seperti `no_sip` sebaiknya disimpan sebagai `NULL`, bukan string kosong, agar tidak melanggar kendala unik.
6. Nomor antrian sebaiknya dibuat server (nomor berikutnya per jadwal dan tanggal), bukan diketik pengguna; kendala unik hanya menolak, tidak menghasilkan nomor.
7. Data `pasien` memuat NIK dan rekam medis: endpoint wajib dilindungi autentikasi dan hak akses per peran, serta dipatuhi aturan pelindungan data pribadi.

10. Tugas Mandiri

1. Tambahkan validasi: nama poli minimal 3 karakter (Bagian A).
2. Tambahkan indikator "Tidak ada koneksi" saat server tidak dapat dijangkau, dengan mengubah `request()` pada `crudService.js` agar membedakan galat jaringan dari galat server.
3. Ganti field **ID Jadwal Praktik** pada form pendaftaran menjadi pilihan (`type: 'select'`) yang memuat data `jadwal_praktik` beserta nama dokter dan poli.
4. Tambahkan kolom pencarian pada layar Pasien (berdasarkan nama atau No. RM) di dalam `makeListScreen`, lalu terapkan ke semua daftar.
5. Buat layar **Detail Rekam Medis** yang menampilkan data pasien, dokter, diagnosa, dan seluruh resep dalam satu halaman.

Rubrik penilaian:

Komponen	Bobot
Bagian A: Read, Create + validasi, Update, Delete + konfirmasi	25%
Layanan dan komponen generik berfungsi (Langkah 13 sampai 16)	20%
Lima tabel Bagian B: CRUD berjalan dan validasi tepat	25%
Penanganan relasi (pilihan dari API, RESTRICT, CASCADE, kendala unik)	15%
Tugas mandiri	10%
Kerapian kode dan struktur folder	5%

11. Referensi

- React Native Team. (2024). *React Native Documentation*. https://reactnative.dev/docs/getting-started
- Expo. (2024). *Expo Documentation*. https://docs.expo.dev
- React Navigation. (2024). *Native Stack Navigator*. https://reactnavigation.org/docs/native-stack-navigator
- MDN Web Docs. (2024). *Using the Fetch API*. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
- Eisenman, B. (2016). *Learning React Native* (2nd ed.). O'Reilly Media.
- Oracle. (2024). *MySQL 8.0 Reference Manual: FOREIGN KEY Constraints*. https://dev.mysql.com/doc/refman/8.0/en/create-table-foreign-keys.html
- Martin, R. C. (2008). *Clean Code*. Prentice Hall. (prinsip DRY dan pemisahan tanggung jawab)

12. Bug Handling

export { default } from



Solution

1. **Cek Node.** Expo SDK 58 mensyaratkan Node **22.13 atau lebih baru** (atau 24.3+). Jalankan node -v. Jika lebih rendah, pasang Node 22 LTS dulu.
2. **Lepas CLI lama:** npm uninstall -g expo-cli (abaikan jika tidak terpasang).
3. **Ganti dependencies di package.json** dengan blok yang sudah saya uji bundel web dan Android-nya:

```
"dependencies": { "@expo/metro-runtime": "~58.0.14",
"@react-navigation/native": "^7.5.1", "@react-
navigation/native-stack": "^7.21.0", "expo": "~58.0.14",
"expo-constants": "~58.0.0", "expo-status-bar": "~58.0.3",
"react": "19.3.0", "react-dom": "19.3.0", "react-native":
"0.88.0", "react-native-safe-area-context": "~5.10.1",
"react-native-screens": "~4.30.0", "react-native-web":
"^0.21.4" }
```

4. **Bersihkan dan pasang ulang**, di folder proyek

```
rm -rf node_modules package-lock.json
npm install npx expo --version # harus tampil 58.x
npx expo start --web --clear
```